What these three networked systems have in common?

Figure: Airline transportation

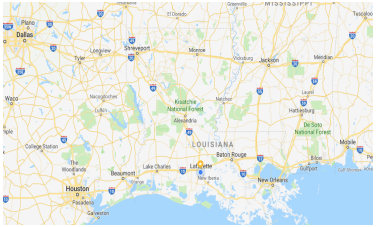


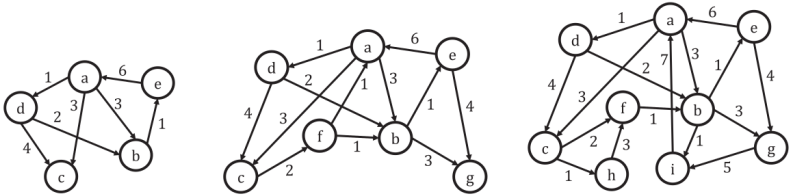
Figure: Google map

Figure: Bitcoin

What these three networked systems have in common?

- ✓ Graphs
- ✓ Time dependent
- ✓ Evolving
- ✓ Streaming

Why Called Evolving & Streaming Graphs?



(a) Snapshot G_1 at t_1. (b) Snapshot G_2 at t_2. (c) Snapshot G_3 at t_3.

KickStarter: Fast and Accurate Computations on Streaming Graphs via Trimmed Approximations

Keval Vora [1], Rajiv Gupta [1], and Guoqing Xu [2]

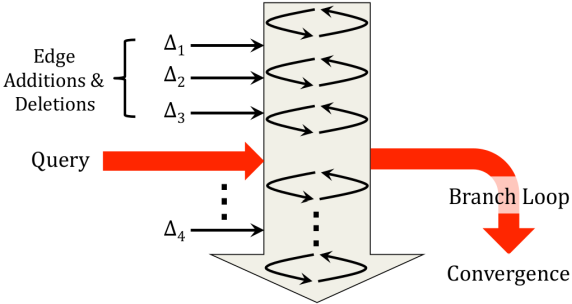
Md. Enamul Haque [3]

[1] University of California, Riverside, CA
[2] University of California, Irvine, CA
[3] School of Computing and Informatics
University of Louisiana, Lafayette, LA

April 19, 2018

Streaming Graph Processing

- ✓ Graph changes rapidly as computation proceeds
- ✓ Incremental processing
- ✓ Maintain "profitable" approximation



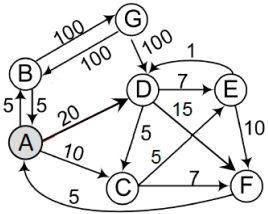
Shao, Xiaogang Shi Bin Cui Yingxia, and Yunhai Tong, Tornado: A System For Real-Time Iterative Analysis Over Evolving Data. , SIGMOD 2016.

The Good, the Bad and the Ugly

Correctness and performance

The Good Scenario

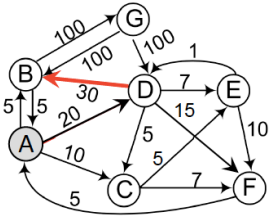
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	5	10	20	7	15	5

The Good Scenario

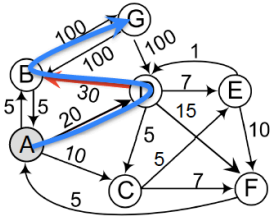
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	5	10	20	7	15	5
Add D → B						

The Good Scenario

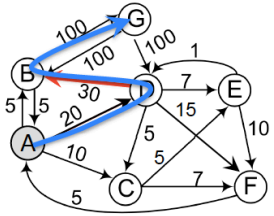
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	5	10	20	7	15	5
Add D → B						

The Good Scenario

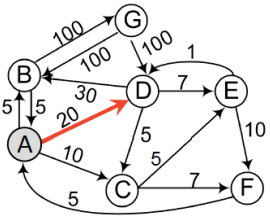
SSWP $v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$



A	B	C	D	E	F	G
∞	5	10	20	7	15	5
Add D → B						
∞	20	10	20	7	15	5
∞	**20**	10	20	7	15	**20**

The Bad Scenario

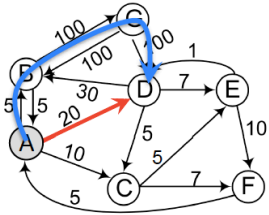
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						

The Bad Scenario

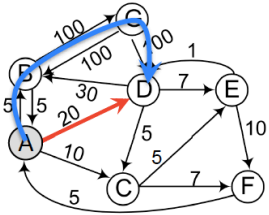
SSWP $$v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A $\rightarrow$ D						

The Bad Scenario

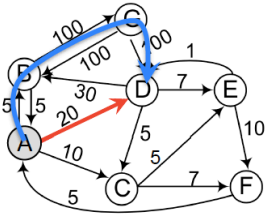
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
...						
∞			20			

The Bad Scenario

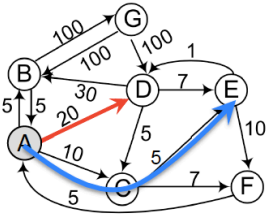
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
...						
∞	20		20			20

The Bad Scenario

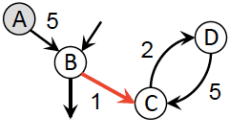
$$\textbf{SSWP} \quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$$



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
...						
∞	20	10	20	7	7	20

The Ugly Scenario

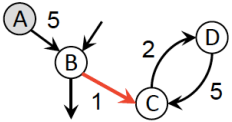
SSSP $$v.path \leftarrow \min_{e \in \text{inEdges}(v)} (e.source.path + e.weight)$$



A	B	C	D
0	5	6	8
Delete B → C			

The Ugly Scenario

SSSP $$v.path \leftarrow \min_{e \in \mathbf{inEdges}(v)} (e.source.path + e.weight)$$



A	B	C	D
0	5	6	8
Delete B → C			
0	5	6	8
0	5	13	8
0	5	13	15
0	5	20	15
0	…	…	…
0	5	MAX	MAX

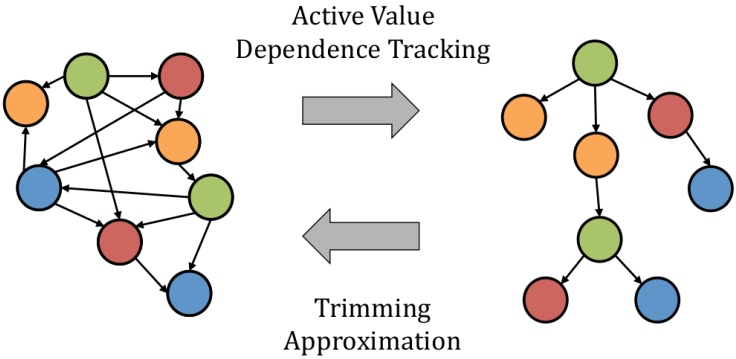
The Good, the Bad, and the Ugly

Edge Deletions

KickStarter

Maintain value dependences during computation

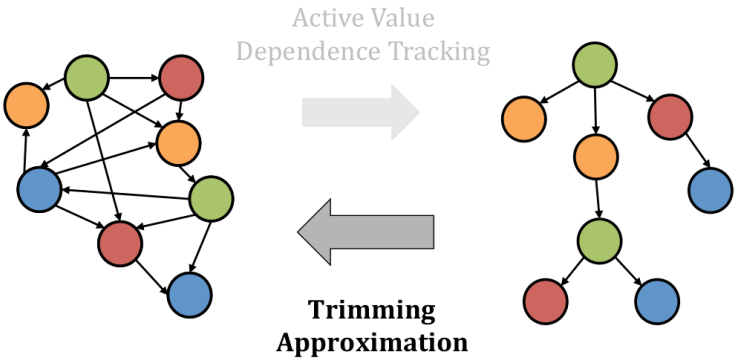
a $\xrightarrow{LT}$ b iff b's value resulted from a



KickStarter

Maintain value dependences during computation

a $\xrightarrow{LT}$ b iff b's value resulted from a



Vertex-centric Trimming Algorithm

Algorithm 1 Vertex-centric trimming algorithm.

```
function TRIM(Vertex v)
    ▷ Construct the subset of incoming edges
    v^s ← ∅
    for e ∈ INCOMINGEDGES(v) do
        if e.source.level ≤ v.level then
            v^s.insert(e)
        end if
    end for
    incomingSet ← CONSTRUCTSUBSET(v.value, v^s)

    ▷ Find safe alternate value
    v.newValue ← VERTEXFUNCTION(incomingSet)

    ▷ Continue trimming if required
    continueTrim ←
            SHOULDPROPAGATE(v.value, v.newValue)
    if continueTrim = true then
        SCHEDULECHILDREN(v)
    end if

    v.value ← v.newValue
    v.UPDATE(incomingSet)
end function
```

Monotonic Graph Algorithms

Vertex values exhibit increasing/decreasing values

- ShortestPaths (SSSP)
- ConnectedComponents
- MinimumSpanningTree
- BreadFirstSearch
- FacilityLocation

- WidestPaths (SSWP)
- Reachability

Monotonic Graph Algorithms

JOURNAL OF MATHEMATICAL ANALYSIS AND APPLICATIONS **54**, 157–172 (1976)

Stability and Convergence of Monotonic Algorithms*

KARL L. NICKEL

Institut für Angewandte Mathematik, Universität Freiburg, 78 Freiburg i. Br., Federal Republic of Germany

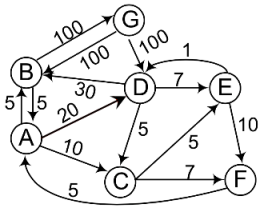
Submitted by W. F. Ames

Monotonic (numerical) algorithms in a partially ordered metric space are considered. Using a termination criterion that is appropriate to the monotonicity, one finds that consistency and local stability imply convergence. Applications to interval analysis are presented. The numerical differentiation of a function is treated as an example.

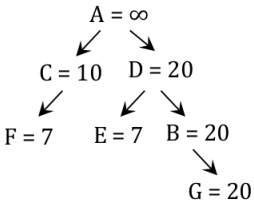
Trimming via Value Dependence

Maintain value dependences during computation

a $\xrightarrow{LT}$ b iff b's value resulted from a



A	B	C	D	E	F	G
∞	20	10	20	7	7	20

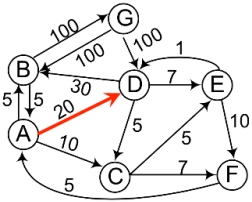


SSWP $\quad v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$

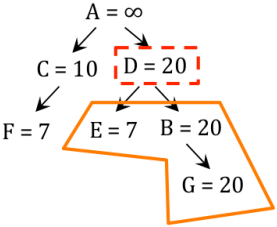
Trimming via Value Dependence

Compute safe approximations

Can be done using the same vertex function



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						

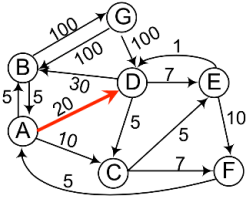


SSWP $v.path \leftarrow \max_{e \in \mathbf{inEdges}(v)} (\min(e.source.path, e.weight))$

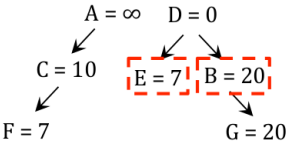
Trimming via Value Dependence

Compute safe approximations

Can be done using the same vertex function



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
∞	20	10	**0**	7	7	20

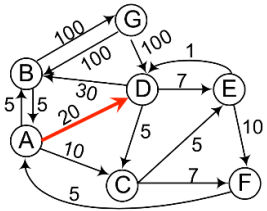


SSWP $v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$

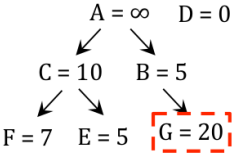
Trimming via Value Dependence

Compute safe approximations

Can be done using the same vertex function



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
∞	20	10	0	7	7	20
∞	5	10	0	5	7	20

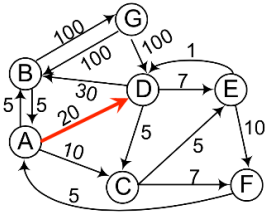


SSWP $v.path \leftarrow \max_{e \in \text{inEdges}(v)} (\min(e.source.path, e.weight))$

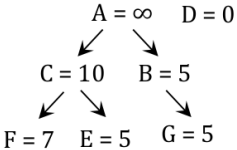
Trimming via Value Dependence

Compute safe approximations

Can be done using the same vertex function



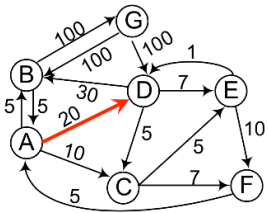
A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
∞	20	10	0	7	7	20
∞	5	10	0	5	7	20
∞	5	10	0	5	7	5
Trimming Complete						



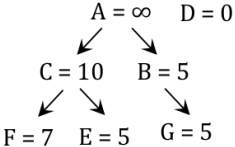
Trimming via Value Dependence

Compute safe approximations

Can be done using the same vertex function



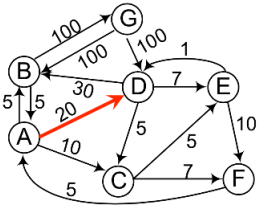
A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
∞	20	10	0	7	7	20
∞	5	10	0	5	7	20
∞	**5**	10	**0**	**5**	7	**5**
Trimming Complete						



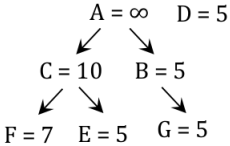
Trimming via Value Dependence

Compute safe approximations

Can be done using the same vertex function



A	B	C	D	E	F	G
∞	20	10	20	7	7	20
Delete A → D						
∞	20	10	0	7	7	20
∞	5	10	0	5	7	20
∞	5	10	0	5	7	5
Trimming Complete						
∞	5	10	5	5	7	5



Experimental Setup

- ✓ 16-node EC2 cluster: 8-core/16GB nodes
- ✓ Monotonic algorithms

Bad Scenario	SingleSourceWidestPaths (SSWP) ConnectedComponents (CC)
Ugly Scenario	SingleSourceShortestPaths (SSSP) BreadthFirstSearch (BFS)

Experimental Setup

- ✓ Streaming graph datasets constructed using [SIGMOD' 16]
- ✓ Fixed point achieved at 50% edges
- ✓ Remaining edges treated as edge additions
- ✓ Rate of update stream (100K-1M updates per query)
- ✓ Edge deletion ratio (10-50%)

Graphs	#Edges	#Vertices
Friendster (FT) [13]	2.5B	68.3M
Twitter (TT) [6]	2.0B	52.6M
Twitter (TTW) [20]	1.5B	41.7M
UKDomain (UK) [5]	1.0B	39.5M
LiveJournal (LJ) [3]	69M	4.8M

Figure: Real world input graphs

Trimming for Correctness

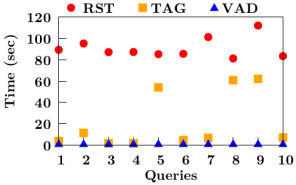
		LJ	UK	TTW	TT	FT
SSWP	**RST**	7.48-10.16 (8.59)	81.22-112.01 (90.75)	94.18-102.27 (99.28)	170.76-183.11 (176.87)	424.46-542.47 (487.04)
	TAG	11.57-14.71 (13.00)	1.73-62.1 (21.42)	27.38-125.91 (71.26)	262.88-278.42 (270.29)	474.64-550.25 (510.52)
	VAD	3.51-5.5 (4.48)	1.17-1.18 (1.17)	21.54-34.38 (27.55)	66.85-130.84 (75.88)	113.3-413.51 (143.72)
CC	**RST**	6.43-7.93 (7.19)	133.92-166.33 (148.80)	105.16-111.46 (107.54)	113.92-126.35 (126.35)	212.43-230.26 (221.05)
	TAG	10.98-12.81 (11.86)	170.91-203.54 (183.93)	176.91-201.12 (185.84)	193.77-249.93 (208.90)	331.79-386 (360.34)
	VAD	4.89-5.85 (5.30)	1.81-7.75 (4.37)	31.78-33.24 (32.54)	21.98-22.58 (22.29)	38-39.36 (38.56)

Figure: Trimming for correctness: query processing time (in sec) for SSWP and CC, shown in the form of min-max (average).

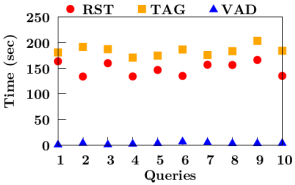
		LJ	UK	TTW	TT	FT
SSWP	**TAG**	3.1M-3.2M (3.1M)	8.9K-9.7M (4.1M)	1.1K-29.5M (13.4M)	28.5M-28.6M (28.6M)	49.5M-49.5M (49.5M)
	VAD	20.1K-90.8K (60.8K)	2.9K-93.0K (33.4K)	1.0K-4.5K (2.3K)	2.4K-1.1M (106.4K)	20.7K-13.6M (1.3M)
CC	**TAG**	3.2M-3.2M (3.2M)	25.9M-25.9M (25.9M)	31.3M-31.3M (31.3M)	32.2M-32.2M (32.2M)	52.1M-52.1M (52.1M)
	VAD	1.1K-3.1K (1.9K)	320-1.6K (1.0K)	116-463 (212)	241-463 (344)	294-478 (374)

Figure: Trimming for correctness: # reset vertices for SSWP and CC (the lower the better) in the form of min-max (average).

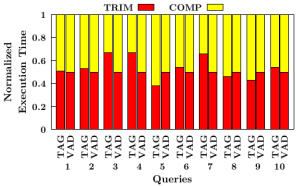
Trimming for Correctness



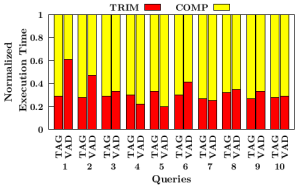
(a) SSWP on UK



(b) CC on UK



(c) SSWP on UK



(d) CC on UK

Trimming for Performance

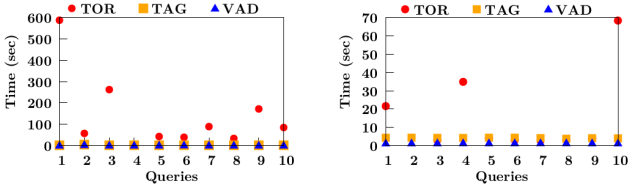
		LJ	UK	TTW	TT	FT
SSSP	TOR	1.27-102.88 (39.10)	2.84-119.03 (24.90)	17.62-131.9 (112.57)	42.13-584.64 (190.78)	90.59-179.83 (163.99)
	TAG	3.25-4.49 (3.97)	2.03-2.94 (2.19)	46.06-52.5 (48.96)	98.59-118.23 (105.73)	131.22-150.16 (142.60)
	VAD	2.12-3.22 (2.55)	1.33-1.5 (1.41)	28.68-32.33 (30.21)	41.35-48.65 (44.19)	93.74-101.67 (97.22)
BFS	TOR	1.17-77.05 (7.17)	1.24-588.09 (142.55)	23.94-1015.76 (199.23)	55-283.71 (120.45)	190.52-2032.38 (881.17)
	TAG	3.47-4.43 (3.88)	1.81-5.14 (1.97)	51.08-58.3 (54.36)	110.75-192.71 (127.54)	143.21-334.07 (166.60)
	VAD	1.96-3.37 (2.59)	1.21-3.88 (1.42)	32.02-34.86 (32.96)	69.43-91.88 (74.27)	107.4-136.73 (114.56)

Figure: Query processing times (in sec) for SSSP and BFS in the form: min-max (average).

		LJ	UK	TTW	TT	FT
SSSP	TAG	8.2K-59.8K (25.9K)	4.1K-193.4K (36.4K)	19.7K-183.7K (89.4K)	6.2K-196.7K (51.5K)	19.8K-31.2K (25.4K)
	VAD	1.7K-40.1K (7.0K)	2.9K-52.2K (16.6K)	2.1K-77.7K (19.6K)	836-110.9K (11.1K)	4.5K-12.5K (8.0K)
BFS	TAG	10.8K-354.5K (79.0K)	1.3K-483.0K (35.5K)	20.9K-1.2M (457.6K)	44.2K-8.6M (1.1M)	19.1K-4.5M (469.8K)
	VAD	5.5K-116.6K (36.4K)	3.2K-469.9K (41.2K)	860-3.1K (1.6K)	742-1.4K (1.1K)	2.7K-5.2K (3.4K)

Figure: Number of reset vertices for SSSP and BFS in the form: min-max (average).

Trimming for Performance



(a) SSSP on UK

(b) BFS on UK

Figure: Time taken to compute query results by TAG and VAD

Summary

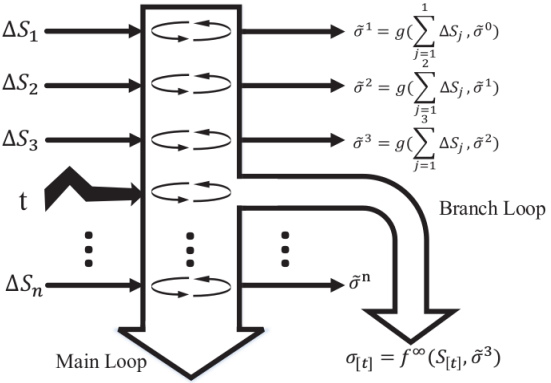
- ✓ Incremental processing in presence of edge deletions
- ✓ Trimming approximation
  - ⋆ Reuse safe and profitable values
- ✓ Active dependence tracking based trimming
  - ⋆ Up to 8.5-23.7× speedups

Questions?

Evolving Graphs

1. How various structural characteristics evolve over time? Leskovec et al. [2005]
2. How user interactions and activities change over time? Wilson et al. [2009]
3. how an author's influence changes in time? Radicchi et al. [2009]

Streaming Graph Processing



How TORNADO works?

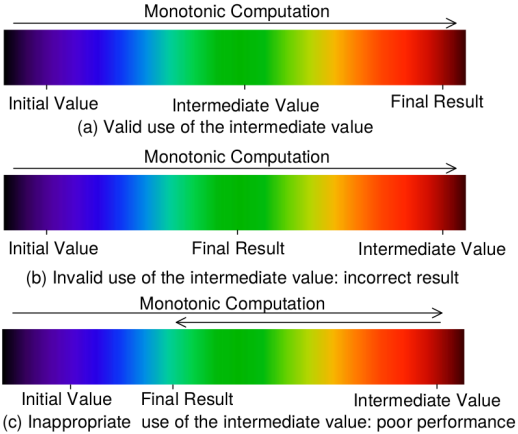


Figure: Three different scenarios w.r.t. the use of intermediate values after an edge update.